



Lecture 04 : Modules and Packages

A module is a file containing Python code that can be re-used in other Python code files.

Check the following four variations for importing modules from packages:

- import [package]
- import [package] as [other_name]
- from [package] import [module]
- from [package] import [module] as [other_name]

Python Libraries

There are libraries already included and others we have to install before being able to import and use them.

The so called "Standard Library" doesn't need to be installed separately and it includes modules like copy, os and shutil, of which we will talk about later in this section.

Numpy and Pandas are external libraries that you need to install before using them.

All these libraries offer you great functions that are ready to be used once imported and which will save you a lot of time and give you new opportunities and functions to work with.

Standard Library:

As you learned, when writing python code you will often import some packages in order to make use of their functions. While there are many packages that you have to install separately, your Python installation comes with a bunch of packages already included. These are called the Python Standard Library.

Packages that are part of the Standard Library are also often called inbuilt packages, as opposed to external packages.

The standard library consists of packages that are commonly used, and you've already used some of them.

If you want to look at all inbuilt packages, here's the official list: [Check this out](#)

COPY, MATH & TIME:

This section will briefly present three inbuilt packages that can be very useful.

The **copy** module is part of the standard library of python and offers the user functions to copy objects without references to the original object.

Assignment statements in Python do not copy objects, they create bindings between a target and an object. For collections that are mutable or contain mutable items, a copy is sometimes needed so one can change one copy without changing the other. This module provides generic shallow and deep copy operations.

```
In [ ]: animals = ['lion', 'tiger', 'cheetah']
       large_cats = animals
       large_cats.append('Leopard')

In [ ]: large_cats

In [ ]: animals

In [ ]: import copy
       animals = ['lion', 'tiger', 'cheetah']
       large_cats = copy.copy(animals)
       large_cats.append('Leopard')

In [ ]: large_cats

In [ ]: animals

math The math package includes - you guessed it - mathematical stuff: it provides mathematical constants, e.g. pi or e, as well as mathematical functions, like log or cos, and also some utilities like ceil and floor.
You will find some examples:

In [ ]: import math
       print(math.log(5, 2))

In [ ]: print(math.pi)

In [ ]: print(math.floor(2.6))

In [ ]: print(math.ceil(2.6))

time The time package deals with time and timing-related things (e.g. timing how fast a function is, but also which year it is). If you're wondering why there's this entire module as well as the datetime and calendar packages; it's because time is quite a tricky thing to properly describe in programming.

In [ ]: import time
       print(time.time())
       # Output: These are the seconds that have passed since January 1st 1970 00:00:00

In [ ]: # The script will now pause for 2 seconds
       time.sleep(2)
       print("hi")

In [ ]: # Here's how we can time how long a function takes:
       def my_function():
           for n in range(1000000):
               n += 2 * n

       start_time = time.time()
       my_function()

       print('{:.4f} seconds'.format(time.time() - start_time))

time object

In [ ]: now = time.localtime()
       print(now)

In [ ]: print(now.tm_year)
```

EXERCISES 03 - PART ONE

HOMEWORK 02

SELF-STUDY

Numerical Computing with NumPy

In Python we can use **lists** to store and manipulate sequences of objects, any objects.

While that is very convenient for us it comes at a cost of time and memory.

In this example we create 1,000,000 integers:

```
In [ ]: import random
       measurements = [random.randint(150, 200) for _ in range(1_000_000)]

In [ ]: measurements[:10]

and computes their mean:

In [ ]: list_time = %timeit -o sum(measurements) / len(measurements)

Because python doesn't know that our list only contains integers, it has to check everytime it adds values together whether the objects actually support addition. That's why sum takes "so long".

If we could tell the interpreter that we are only adding integers, we could skip all that typechecking and speed up the operation.

For this purpose, numpy was invented.

To use numpy we have to import it. The import is usually aliased as np so we have to type less later on.

In [ ]: import numpy as np

Numpy's standard datatype is the ndarray (which stands for n-dimensional array). In the simplest case, numpy array can be created from list.

In [ ]: measurements_array = np.array(measurements)

In [ ]: measurements_array

Now we can use the built-in mean function provided by Numpy:

In [ ]: numpy_time = %timeit -o np.mean(measurements_array)

As we can see, using Numpy significantly speeds up our computation, making it

In [ ]: print(f'({list_time.average / numpy_time.average}) times faster')
```

Numpy Arrays

creating arrays from lists

As we already saw we can create Numpy Arrays from lists

```
In [ ]: np.array([10,2,35])

creating arrays using Numpy functions

Whenever we don't want to create an array from specific values like [10, 2, 35] we can use Numpy's utility functions.

These are also faster than Python's built-in functions. (e.g. np.arange works like range)

In [ ]: list(range(5))

In [ ]: np.arange(start=2, stop=14, step=2)

np.linspace

Creating an array with a certain number of values in a certain interval:

In [ ]: np.linspace(start=-5, stop=5, num=9)

np.zeros and np.ones

Creating an array with only zeros or ones:

In [ ]: np.zeros(3), np.ones(3)

Both take a shape argument that lets us create multidimensional arrays.

In [ ]: np.zeros((2, 2))

In [ ]: np.ones((2,3))
```

Anatomy of arrays

dtype

Returns data type of the array. Arrays can contain bools, ints, unsigned ints, floats or complex numbers of various byte sizes.

They can also store strings or Python objects, but that has very few use cases.

```
In [ ]: values = [0, 1, 2, 3, 4]
       int_arr = np.array(values, dtype='int')
       int_arr, int_arr.dtype

If the dtype does not match the given values, numpy will cast everything to that data type.

In [ ]: bool_arr = np.array(values, dtype='bool')
       bool_arr, bool_arr.dtype

If no explicit data type is given, numpy will choose the "smallest common denominator".

In the following example, everything becomes a float, as ints can be represented as floats, but not vice versa.

In [ ]: values = [0, 1, 2.5, 3, 4]
       float_arr = np.array(values)
       float_arr, float_arr.dtype
```

shape and ndim

shape is very important for keeping track of arrays with more than one dimension. It is a tuple with the number of elements in each dimension.

ndim is just the number of dimensions in total.

1D

```
In [ ]: values = [1, 2, 3, 4, 5]
       one_dim_arr = np.array(values)
       one_dim_arr

In [ ]: one_dim_arr.shape

In [ ]: one_dim_arr.ndim

2D

In [ ]: values = [[1, 2, 3, 4, 5],
               [1, 2, 3, 4, 1],
               [1, 2, 3, 4, 2]]
       two_dim_arr = np.array(values)
       two_dim_arr

In [ ]: two_dim_arr.shape

In [ ]: two_dim_arr.ndim

In [ ]: two_dim_arr[1, 3]

In [ ]: two_dim_arr[1,3] = 10
       two_dim_arr
```

3D

```
In [ ]: values = [[[1, 2, 3, 4]] * 3] * 6
       three_dim_arr = np.array(values)
       three_dim_arr

In [ ]: three_dim_arr.shape

In [ ]: three_dim_arr.ndim

In [ ]: three_dim_arr[1,1,1]
```

Numpy Functions - what can we do with arrays

reshape

```
In [ ]: a = np.arange(start=2, stop=14)
       print(a.shape)
       a

In [ ]: a.reshape(3, 4)

-1 as axis automatically figures out the size of the dimension which we didn't explicitly specify.

In [ ]: a.reshape(2, -1)

In [ ]: a.reshape(2, -1).shape
```

Comparing Arrays

```
In [ ]: a = np.zeros((3,3))
       b = np.zeros((3,3))
       a == b

In [ ]: a[0,0] == 0.000000000000001
       a == b

In [ ]: np.isclose(a, b)
```

Mathematical operations

Numpy contains a lot of mathematical functions that operate on arrays in a vectorized manner. That means that they are applied to each element, without explicit for-loops. Vectorized functions are called **ufuncs** (universal functions) in Numpy.

```
In [ ]: arr = np.arange(1, 10)
       arr

In [ ]: arr + 3

In [ ]: arr * arr

In [ ]: arr + (arr**2)

In [ ]: arr - arr

In [ ]: arr / arr

In [ ]: arr ** 2
```

Using **@** you can even do matrix multiplication. In the case of 1d arrays, this is the inner product between two vectors.

```
In [ ]: arr @ arr

In [ ]: # That's the same as
       np.sum(arr * arr)

In [ ]: np.log(arr)

In [ ]: np.log2(arr)

In [ ]: np.exp(arr)

In [ ]: np.sin(arr)
```

Aggregation functions

Aggregation functions are functions that reduce the dimensionality of an array. They provide an **axis** argument, to specify which dimension to reduce.

```
In [ ]: two_dim_arr = np.random.randint(0, high=20, size=(3, 4))
       two_dim_arr

If just the array is passed, the aggregation operation is performed over the whole array.

In [ ]: np.min(two_dim_arr)

The optional axis argument allows us to specify, which dimension should be aggregated. You can think of it as the operation being applied to all entries that are obtained by keeping the indices in all dimensions fixed except for the axis dimension. Let's look at the result of the minimum operation with axis=0:
```

```
In [ ]: np.min(two_dim_arr, axis=0)

In [ ]: np.max(two_dim_arr)

In [ ]: np.sum(two_dim_arr)

In [ ]: np.average(two_dim_arr, axis=0)
```

Combining arrays

There are many ways to combine existing arrays, like **np.append**, **np.concatenate** and **np.stack**. However, these operations always require the whole array to be copied. Therefore, it often makes more sense to allocate an array of the size you need later upfront and then just fill the respective parts.

concatenate

```
In [ ]: a = np.arange(10)
       b = np.arange(10)[1:-1]

In [ ]: a

In [ ]: b

In [ ]: # Needs a sequence (in this case a tuple) of array-likes
       np.concatenate((a,b))
```

append

uses **concatenate** internally

```
In [ ]: # Needs exactly two array-likes
       np.append(a, b)

stack

For higher-dimensional arrays, other functions are useful

In [ ]: np.stack((np.arange(10), np.arange(10)))
```

Masking

Logical arrays, i.e. arrays containing boolean values, can be used to index other arrays. These logical arrays are then called masks. This is especially useful to index based on logical conditions.

```
In [ ]: # A simple integer array.
       arr = np.arange(1, 6)
       arr

In [ ]: # A boolean array of the same shape as arr.
       mask = np.array([True, False, True, False, True])
       mask

Using the masked array, it's possible to return only the element where the mask is true.

In [ ]: arr[mask]

In [ ]: arr < 5

In [ ]: arr[arr < 3]
```

Indexing

```
In [ ]: large_two_dim_arr = np.arange(81).reshape((9, 9))
       large_two_dim_arr

the syntax works as [row, column], to break it down more, [row_start : row_end, col_start : col_end]

In [ ]: large_two_dim_arr[:, 2]

In [ ]: large_two_dim_arr[1, 2:7]

with Standard slicing (start, stop, step) works as expected.

In [ ]: large_two_dim_arr[:, 2:7:2]
```

```
In [ ]: 
```

```
In [ ]: 
```